EXPERIMENT 4

```
print("J.THASLIMA NASREEN - 192424152")
# Artificial Neural Network using Backpropagation Algorithm

import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# ------------------------------
# Load the Iris Dataset
# ------------------------------
iris = load_iris()

X = iris.data
y = iris.target

# One-Hot Encoding of Target
encoder = OneHotEncoder(sparse_output=False)
y_encoded = encoder.fit_transform(y.reshape(-1, 1))

# Feature Scaling
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split the Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.2, random_state=42
)

# ------------------------------
# Sigmoid Activation Function
# ------------------------------
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of Sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# ------------------------------
# Initialize Neural Network
# ------------------------------
```

```
input_neurons = X_train.shape[1]      # 4
hidden_neurons = 8
output_neurons = y_train.shape[1]     # 3

np.random.seed(42)

W1 = np.random.randn(input_neurons, hidden_neurons)
b1 = np.zeros((1, hidden_neurons))

W2 = np.random.randn(hidden_neurons, output_neurons)
b2 = np.zeros((1, output_neurons))

learning_rate = 0.1
epochs = 5000

# ----------------------------
# Training using Backpropagation
# ----------------------------
for epoch in range(epochs):

    # Forward Propagation
    hidden_input = np.dot(X_train, W1) + b1
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W2) + b2
    predicted_output = sigmoid(final_input)

    # Error Calculation
    error = y_train - predicted_output

    # Backpropagation
    d_output = error * sigmoid_derivative(predicted_output)

    hidden_error = np.dot(d_output, W2.T)
    d_hidden = hidden_error * sigmoid_derivative(hidden_output)

    # Update Weights and Biases
    W2 += learning_rate * np.dot(hidden_output.T, d_output)
    b2 += learning_rate * np.sum(d_output, axis=0, keepdims=True)

    W1 += learning_rate * np.dot(X_train.T, d_hidden)
    b1 += learning_rate * np.sum(d_hidden, axis=0, keepdims=True)

    # Display Loss
```

```
    if epoch % 500 == 0:
        loss = np.mean(np.square(error))
        print(f"Epoch {epoch}, Loss = {loss:.4f}")

# ----------------------------
# Testing the Model
# ----------------------------
hidden_test = sigmoid(np.dot(X_test, W1) + b1)
output_test = sigmoid(np.dot(hidden_test, W2) + b2)

predictions = np.argmax(output_test, axis=1)
actual = np.argmax(y_test, axis=1)

accuracy = np.mean(predictions == actual) * 100

print("\nAccuracy:", round(accuracy, 2), "%")

# ----------------------------
# Display Results
# ----------------------------
flower_names = iris.target_names

results = pd.DataFrame({
    "Actual": [flower_names[i] for i in actual],
    "Predicted": [flower_names[i] for i in predictions]
})

print("\nActual vs Predicted")
print(results.head(10))
```

OUTPUT

```
Actual  Predicted
0       1          1
1       0          0
2       2          2
3       1          1
4       1          1
5       0          0
6       1          1
7       2          2
8       1          1
9       1          1
J.THASLIMA NASREEN - 192424152
```